Dynamic Programming(2)

Soontharee Koompairojn

String Alignment (Edit Distance) problem

- เป็นการเปรียบเทียบความเหมือนกันของตัวอักษรบนสายสตริง 2 สาย ถ้าคะแนนความเหมือนกันมากที่สุดแสดงว่าคล้ายกันมาก
- การแก้ไขและเปรียบเทียบ(edit distance) ของสายสตริง 2 สาย จะต้องมีการแก้ไขน้อยที่สุด
- ความเป็นไปได้ในการแก้ไขสตริง A และ B ให้มีความเหมือนกันมีดังนี้

1. Character `A[i]` and `B[i]` **match** and we do nothing (assume this worth '+2' score),
2. Character `A[i]` and `B[i]` **mismatch** and we replace `A[i]` with `B[i]` (assume '-1' score),
3. We insert a space in `A[i]` (also '-1' score),
4. We delete a letter from `A[i]` (also '-1' score).

For example: (note that we use a special symbol '_' to denote a space)

```
A = 'ACAATCC' -> 'A_CAATCC'          // Example of a non optimal alignment
B = 'AGCATGC' -> 'AGCATGC_'                 // Check the optimal one below
                 2-22--2-             // Alignment Score = 4*2 + 4*-1 = 4
```

- ปัญหานี้เป็นการเปลี่ยนตัวอักษรใน A ให้เท่ากับ B โดยที่ให้ได้คะแนนมากที่สุด
- สมมติมีสตริง A[1..n]
  B[1..m]
- ให้ V(i,j) เป็นคะแนน optimal alignment ระหว่างA[1..i] และ B[1..j]
- ให้ score(C1,C2) เป็นฟังก์ชันที่ส่งกลับค่าคะแนนการ align C1 กับ C2
- จะได้ว่า

Base cases:
$V(0,0) = 0$ // no score for matching two empty strings
$V(i,0) = i \times score(A[i], _)$ // delete substring `A[1..i]` to make the alignment, $i > 0$
$V(0,j) = j \times score(_, B[j])$ // insert spaces in `B[1..j]` to make the alignment, $j > 0$

Recurrences: For $i > 0$ and $j > 0$:
$V(i,j) = max(option1, option2, option3)$, where
$option1 = V(i-1, j-1) + score(A[i], B[j])$ // score of match or mismatch
$option2 = V(i-1, j) + score(A[i], _)$ // delete A_i
$option3 = V(i, j-1) + score(_, B[j])$ // insert B_j

$V(i,0) = i \times score(A[i], _)$ // delete substring `A[1..i]` to make the alignment, $i > 0$
$V(0,j) = j \times score(_, B[j])$ // insert spaces in `B[1..j]` to make the alignment, $j > 0$

Recurrences: For $i > 0$ and $j > 0$:
$V(i,j) = max(option1, option2, option3)$, where
$option1 = V(i-1, j-1) + score(A[i], B[j])$ // score of match or mismatch
$option2 = V(i-1, j) + score(A[i], _)$ // delete A_i
$option3 = V(i, j-1) + score(_, B[j])$ // insert B_j

	_	A	G	C	A	T	G	C
_	0	-1	-2	-3	-4	-5	-6	-7
A	-1							
C	-2		Base Cases					
A	-3							
A	-4							
T	-5							
C	-6							
C	-7							

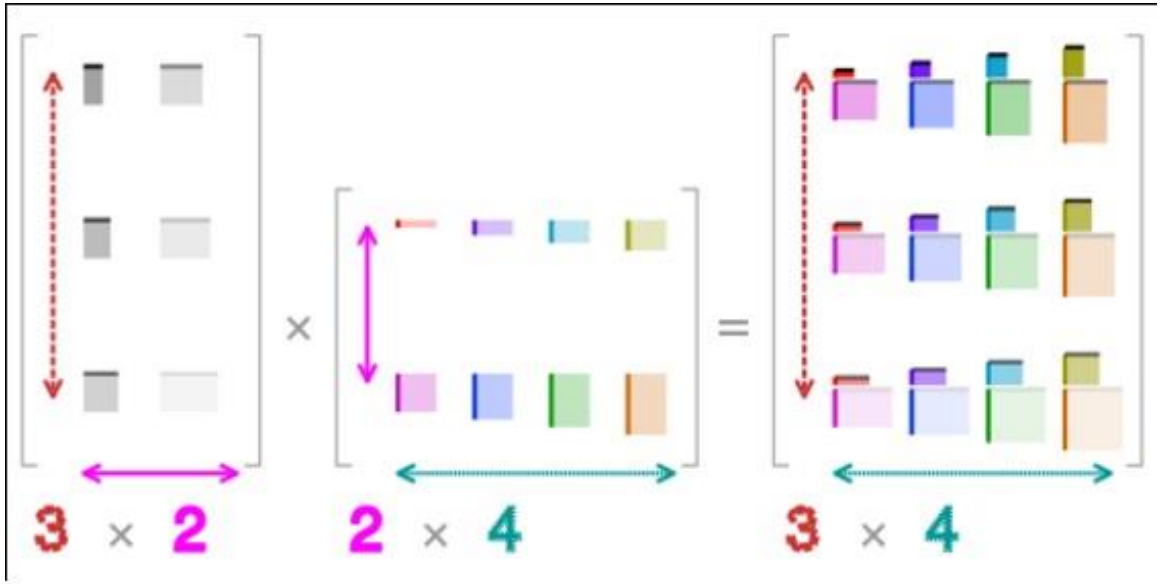
	_	A	G	C	A	T	G	C
_	0	-1	-2	-3	-4	-5	-6	-7
A	-1	2	1	0	-1	-2	-3	-4
C	-2	1	1	3				
A	-3							
A	-4							
T	-5							
C	-6							
C	-7							

	_	A	G	C	A	T	G	C
_	0	-1	-2	-3	-4	-5	-6	-7
A	-1	2	1	0	-1	-2	-3	-4
C	-2	1	1	3	2	1	0	-1
A	-3	0	0	2	5	4	3	2
A	-4	-1	-1	1	4	4	3	2
T	-5	-2	-2	0	3	6	5	4
C	-6	-3	-3	0	2	5	5	7
C	-7	-4	-4	-1	1	4	4	7

Figure 6.1: Example: `A = 'ACAATCC'` and `B = 'AGCATGC'` (alignment score = 7)

ที่มา Steven & Felix, Competitive Programming

Matrix Multiplication



$$\mathbf{A} = \begin{pmatrix} a & b & c \\ p & q & r \\ u & v & w \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} x \\ y \\ z \end{pmatrix},$$

จำนวนครั้งของการคูณ AB =?

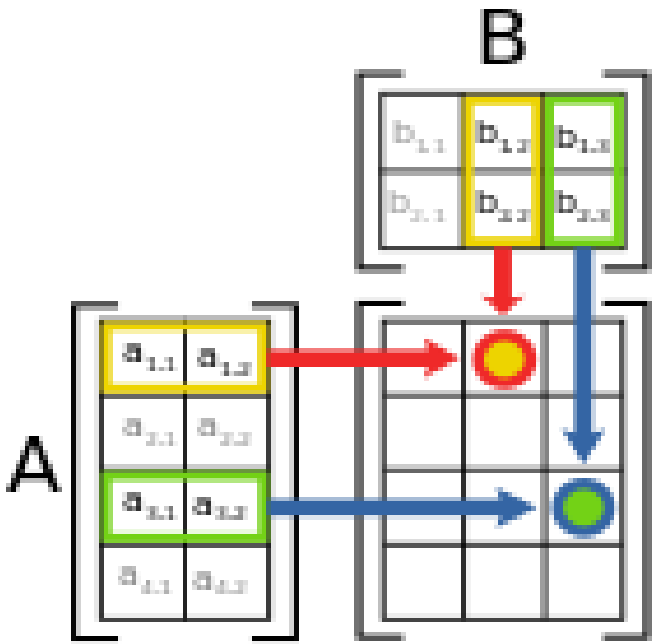
their matrix product is:

$$\mathbf{AB} = \begin{pmatrix} a & b & c \\ p & q & r \\ u & v & w \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} ax + by + cz \\ px + qy + rz \\ ux + vy + wz \end{pmatrix},$$

http://en.wikipedia.org/wiki/Matrix_multiplication

Matrix Multiplication

- ให้ A เป็นเมทริกซ์ขนาด p x q
- ให้ B เป็นเมทริกซ์ขนาด q x r
- ให้ C เป็นเมทริกซ์ผลคูณ AB , ขนาด p x r , จำนวนครั้งการคูณกัน p x q x r ครั้ง



$$c[i,j] = \sum_{k=1}^{q} a[i,k]b[k,j]$$

For $1 \le i \le p$ and $1 \le j \le r$

http://en.wikipedia.org/wiki/Matrix_multiplication

Matrix Multiplication

- Matrix multiplication มีคุณสมบัติ associative.
- สำหรับเมทริกซ์ A, B และ C ใดๆ, A(BC) = (AB)C
- สมมติ A เป็นเมทริกซ์ขนาด r x 2

  B เป็นเมทริกซ์ขนาด 2 x r

  C เป็นเมทริกซ์ขนาด r x 1

- จำนวนการคูณกันของผลคูณเมทริกซ์ A(BC) เป็นเท่าใด?
- จำนวนการคูณกันของผลคูณเมทริกซ์ (AB)C เป็นเท่าใด?

Matrix Chain Multiplication

- กำหนดให้ A chain $\langle A_1 A_2 \ldots A_n \rangle$
- A_i เป็นเมทริกซ์ขนาด $p_{i-1} \times p_i$ โดยที่ $1 \le i \le n$
- **ปัญหา** ให้ใส่วงเล็บแบ่งการคูณย่อยของเมทริกซ์ทั้งหมด ที่ทำให้ จำนวนครั้งของการคูณกันของเมทริกซ์ มี**ค่า<u>น้อยที่สุด</u>**

<u>Method</u>

1. Exhaustive search?
   the run time is exponential in n

   that is, $\binom{2n}{n} / (n+1) = \Omega(4^n / n^{3/2})$

2. Dynamic Programming

Matrix Chain Multiplication

- Developing a dynamic programming algorithm

- กำหนดโครงสร้างของ optimal solution

แบ่งปัญหาเป็นปัญหาย่อย

Let $M = \underbrace{A_1 A_2 \ldots A_k}_{M_1} \underbrace{A_{k+1} A_{k+2} \ldots A_n}_{M_2} = M_1 M_2$

<u>ข้อสังเกต</u> สำหรับค่า k ใดๆ

ถ้าจะให้การแก้ปัญหา เมทริกซ์ M optimal

การแก้ปัญหา เมทริกซ์ M_1 และ M_2 จะต้อง optimal ก่อน

Matrix Chain Multiplication

For $1 \le i \le j \le n$

กำหนดให้ m[i,j] เก็บ minimum cost

(minimum number of multiplications used)

ในการคูณ subchain $A_i\ A_{i+1} \ldots A_j$

กำหนดให้ A_i เป็น เมทริกซ์ขนาด $p_{i-1} \times p_i$ โดยที่ $(1 \le i \le n)$

<u>เริ่มต้นโปรแกรม</u>

ให้นำ ขนาดของเมทริกซ์ทั้งหมดมาใส่ array 1 มิติ P นั่นคือ $p_0\ p_1\ p_2 \ldots p_n$

ตัวอย่าง A1 A เป็นเมทริกซ์ขนาด 3 x 2

A2 B เป็นเมทริกซ์ขนาด 2 x 3

A3 C เป็นเมทริกซ์ขนาด 3 x 1

i=2 pi-1 = p1 = 2

pi = p2 = 3

Array P สำหรับเก็บขนาดเมทริกซ์ ABC

0	1	2	3
3	2	3	1

A B C

สำหรับตัวแบ่ง subchain k ใดๆ ใน $i \le k < j$

ของ เมทริกซ์ subchain $A_i\ A_{i+1} \ldots A_j$

$$A_{i-1}\ \underbrace{A_i\ A_{i+1} \ldots A_k}_{m[i,k]}\ \underbrace{A_{k+1}\ A_{k+2} \ldots A_j}_{m[k+1,j]}$$

optimal solution m[i,j] สำหรับการแบ่งที่ k ใดๆ

มี cost 3 ค่า ที่เกี่ยวข้องคือ

1.ค่า m[i,k] เป็นค่า minimum cost ของการคำนวณ $A_i\ A_{i+1} \ldots A_k$

2.ค่า m[k+1,j] เป็นค่าminimum costของการคำนวณ $A_{k+1}\ A_{k+2} \ldots A_j$

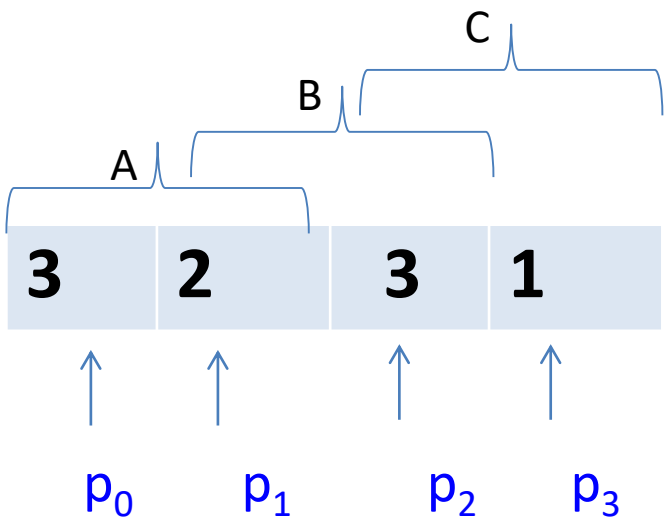
3.ค่า $p_{i-1}\ p_k\ p_j$ ที่ได้จากการคำนวณเมทริกซ์ 2 subchain m[i,k]กับm[k+1,j]

จะได้ว่า minimum cost ของการคูณ subchain $A_i\ A_{i+1} \ldots A_j$ คือ

$$m[i,j] = \min_{i \le k < j} [m[i,k] + m[k+1,j] + p_{i-1} p_k p_j]$$

ตัวอย่าง

Array P สำหรับเก็บขนาดเมทริกซ์ ABC



ให้ i = 1, j =3, k=2

m[1,3] เก็บจำนวนครั้งที่น้อยที่สุดของการคูณ ABC

m[1,2] เก็บจำนวนครั้งที่น้อยที่สุดของการคูณ AB =?

m[3,3] เก็บจำนวนครั้งที่น้อยที่สุดของการคูณ C = 0

$p_0p_2p_3$ ผลของการคูณเมทริกซ์ AB กับ C

m[i,j]	1	2	3
1	0		
2		0	
3			0

สำหรับ p_i และ p_j จาก array 1 มิติข้างบน
สำหรับ array 2 มิติ minimum cost m[i,j]
ในกรณีที่ i = j จะได้ว่า m[i,j] = 0

Matrix Chain Multiplication

ตัวอย่าง : สมมติว่ามี chain ของเมทริกซ์ 4 ตัว

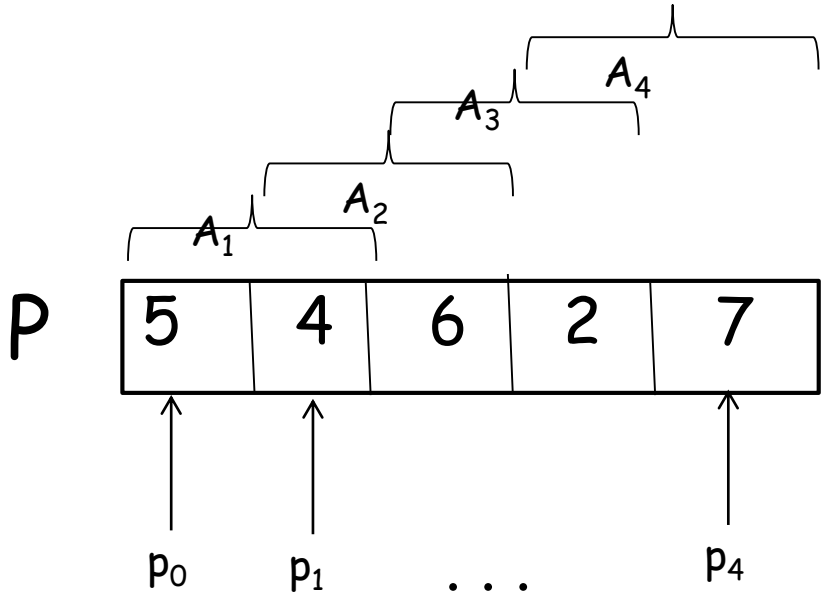
ประกอบด้วย A_1 มีขนาด 5x4 matrix

A_2 มีขนาด 4x6 matrix

A_3 มีขนาด 6x2 matrix

A_4 มีขนาด 2x7 matrix

คำนวณหาจำนวณการคูณกันที่น้อยที่สุดของ chain นี้ นั่นคือ m[1,4] = ?



m[i,j]	1	2	3	4
1	0			
2		0		
3			0	
4				0

Matrix Chain Multiplication

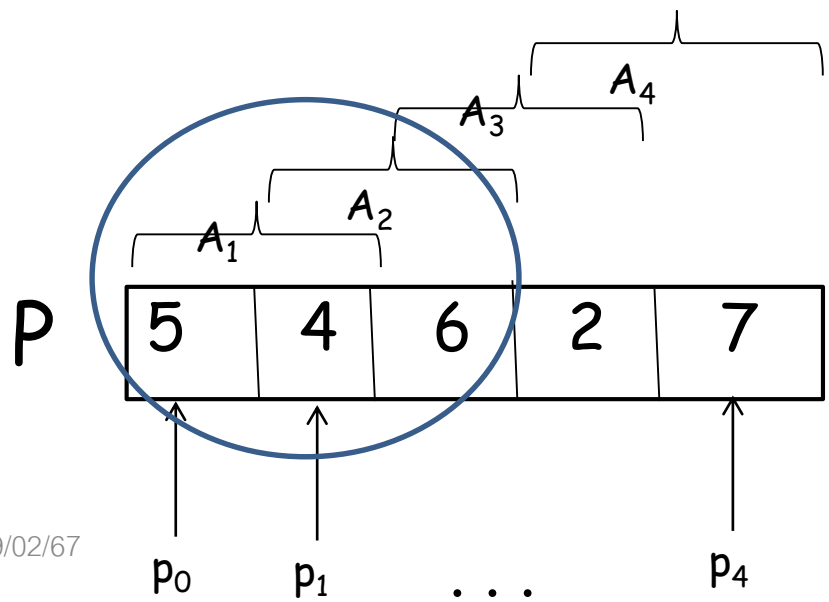
Step1 : Computing m[1,2]

definition $m[i,j] = \min_{i \le k < j} [m[i,k] + m[k+1,j] + p_{i-1}\, p_k\, p_j]$

$$m[1,2] = \min_{1 \le k < 2} [m[1,k] + m[k+1,2] + p_0 p_k p_2$$

$$= m[1,1] + m[2,2] + p_0\, p_1\, p_2$$

$$= 0 \quad + 0 \quad + 5 \times 4 \times 6 = 120$$



m[i,j]	1	2	3	4
1	0	120		
2		0		
3			0	
4				0

Matrix Chain Multiplication

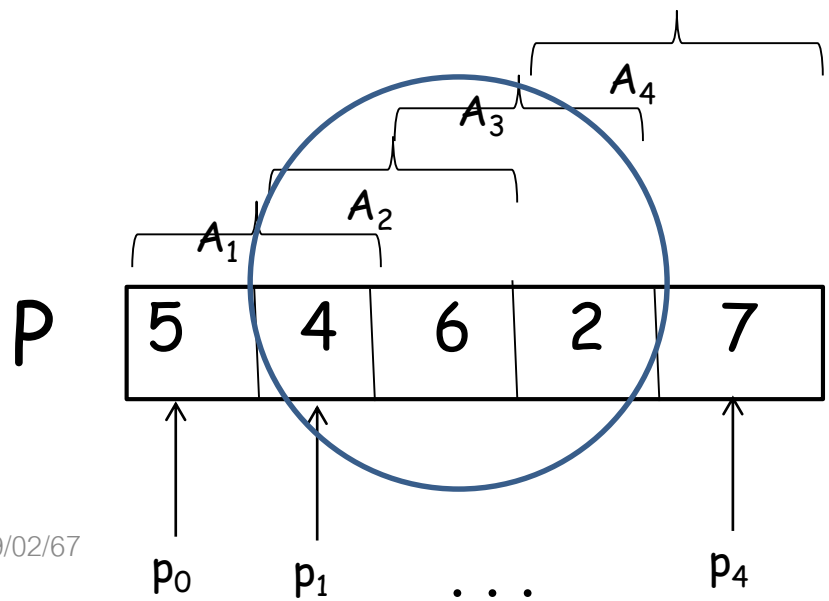
Step 2: Computing m[2,3]

definition $m[i,j] = \min_{i \le k < j} [m[i,k] + m[k+1,j] + p_{i-1}\, p_k\, p_j]$

$$m[2,3] = \min_{2 \le k < 3} [m[2,k] + m[k+1,3] + p_1\, p_k\, p_3]$$

$$= m[2,2] + m[3,3] + p_1\, p_2\, p_3$$

$$= 0 \quad + 0 \quad + 4 \times 6 \times 2 = 48$$



m[i,j]	1	2	3	4
1	0	120		
2		0	48	
3			0	
4				0

Matrix Chain Multiplication

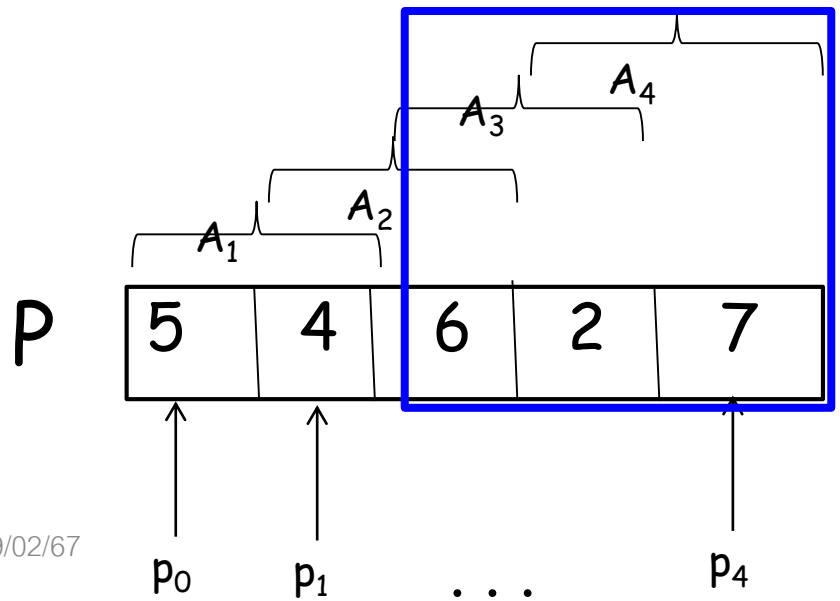
Step 3: Computing m[3,4]

definition $m[i,j] = \min_{i \le k < j} [m[i,k] + m[k+1,j] + p_{i-1}\, p_k\, p_j]$

$m[3,4] =$

$=$

$=$



m[i,j]	1	2	3	4
1	0	120		
2		0	48	
3			0	
4				0

Matrix Chain Multiplication

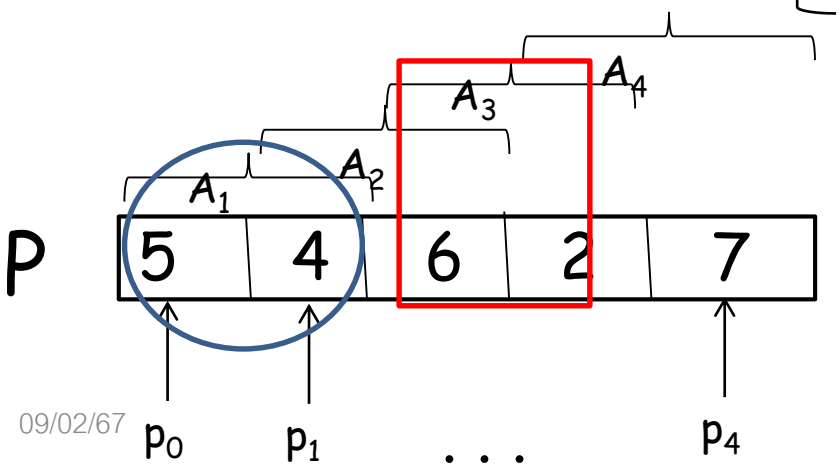
Step 4: Computing m[1,3]

definition $m[i,j] = \min_{i \le k < j} [m[i,k] + m[k+1,j] + p_{i-1}\, p_k\, p_j]$

$$m[1,3] = \min_{1 \le k < 3} [m[1,k] + m[k+1,3] + p_0 p_k p_3]$$

$$m[1,3] = \min \begin{cases} m[1,1] + m[2,3] + p_0 p_1 p_3 \\ m[1,2] + m[3,3] + p_0 p_2 p_3 \end{cases}$$

$$= \min \begin{cases} 0 + 48 + 5 \times 4 \times 2 \\ 120 + 0 + 5 \times 6 \times 2 \end{cases} = 88$$



m[i,j]	1	2	3	4
1	0	120	88	
2		0	48	
3			0	84
4				0

Matrix Chain Multiplication

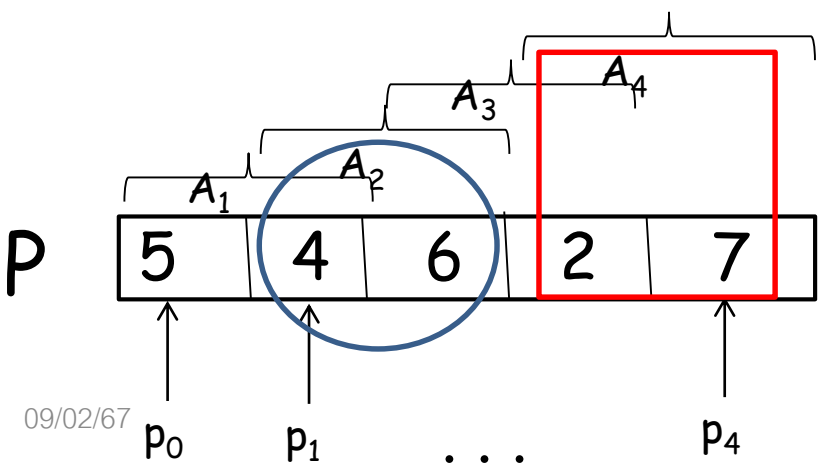
Step 5: Computing m[2,4]

definition $m[i,j] = \min_{i \le k < j} [m[i,k] + m[k+1,j] + p_{i-1}\, p_k\, p_j]$

$$m[2,4] =$$

$$=$$

$$=$$



m[i,j]	1	2	3	4
1	0	120	88	
2		0	48	
3			0	84
4				0

Matrix Chain Multiplication

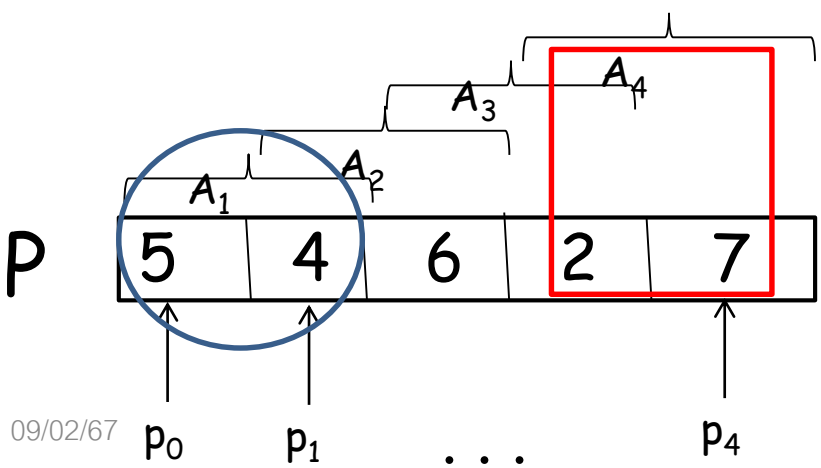
Step 6: Computing m[1,4]

definition $m[i,j] = \min_{i \le k < j} [m[i,k] + m[k+1,j] + p_{i-1}\, p_k\, p_j]$

$$m[1,4] = \min_{1 \le k < 4} [m[1,k] + m[k+1,4] + p_0 p_k p_4]$$

$$= \min \begin{cases} m[1,1]+m[2,4]+ p_0 p_1 p_4 \\ m[1,2]+m[3,4]+ p_0 p_2 p_4 \\ m[1,3]+m[4,4]+ p_0 p_3 p_4 \end{cases}$$

$=$



m[i,j]	1	2	3	4
1	0	120	88	
2		0	48	104
3			0	84
4				0